

STAT:5400 Section 2.1 — A First Look at R

Boxiang Wang

2026-08-31

1 A First Look at R

1.1 An implementation of S language

- S language was developed at AT&T-Bell Labs; first version 1976.
- S-Plus is a commercial version of S (begin in 1987), sold and supported by Insightful Corp.

What S-Plus offered

Bottom line. S-Plus was the paid product built on S. R is the free implementation that replaced it in practice.

- GUI.
- Many formats supported for graphics export and data input/output.
- Runs on Windows, UNIX, Linux (not Macintosh).

1.2 R

- International team of statisticians started developing R in early 1990's.
 - To provide **free and open source** alternative to S-Plus.
 - To provide S implementation on Linux (not supported by S-Plus then); your code can be replicated by users on Windows, Unix, Linux, Mac...
- Easy to [download](#) and install from web sites.
- User-contributed packages expand capabilities and provide support for cutting edge research.
- An active community.
- Facilitate communicating your results: knitr, r markdown, shiny.

- Support of calling high-performance languages: C, C++, Fortran... some connection with Python and Matlab.
- On the other hand, R itself is not very fast and efficient in using memory. Users' coding is sometimes not elegant.

1.3 What R is

- “An integrated suite of software facilities for data manipulation, calculation, and graphics display”

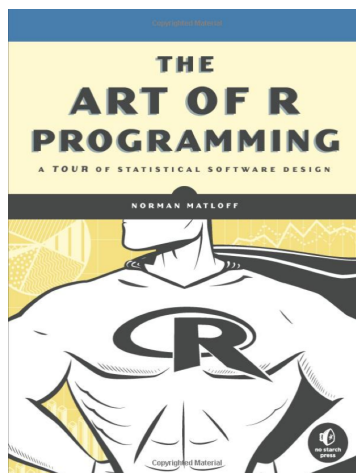
(*An Introduction to R*, Venables, Ripley, and the R Core team).

- The goal of STAT 5400 regarding R:
 - Get familiar with fundamentals of R: data types, operations, object systems.
 - Have extensive hands-on experiences of doing statistical analysis using R, with some touch on modern features in R.
 - Use profiling to find computational bottlenecks and improve by writing a more elegant R code or resorting to C and Fortran.

1.4 Reference

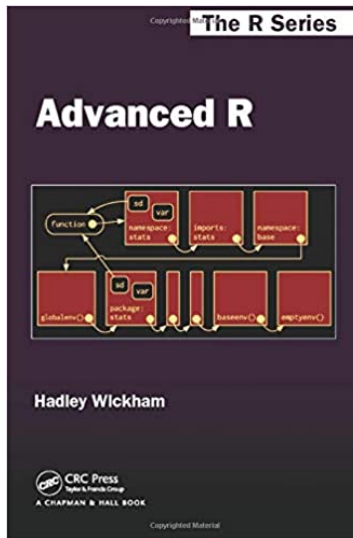
The Art of R Programming: A Tour of Statistical Software Design.

By Norman Matloff.

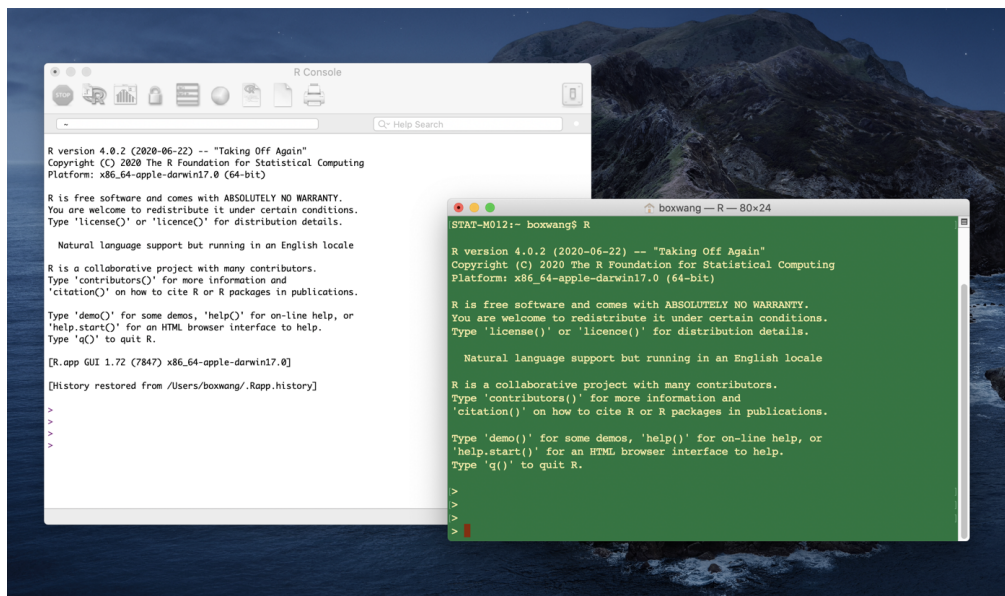


Advanced R.

By Hadley Wickham ([COPSS Presidents' Award 2019](#)).



1.5 Basic operation



```
1 + 1
exp(-2)
sqrt(2 * pi)
```

```
2 / 0
3 / 2 * 3
```

```
print("Hello 5400!")
```

💡 Predict — say the answer out loud before you run it.

```
0.1 + 0.2 == 0.3
all.equal(0.1 + 0.2, 0.3)
.Machine$double.eps
```

i Why FALSE?

Bottom line. 0.1, 0.2 and 0.3 cannot be written exactly in binary, so the sum is off by about $5.6\text{e-}17$. Never compare two computed numbers with `==`. Use `all.equal()`, which allows a tolerance of about $1.5\text{e-}8$ by default. `.Machine$double.eps` is the smallest number that still makes a difference when added to 1. On almost every machine it is $2.220446\text{e-}16$.

💡 Predict — one of these two lines warns you. Which one, and why?

```
1:3 + 1:2
1:4 + 1:2
```

i Which one warns?

The first one warns; the second is silent. Both recycle the shorter vector. R warns only when the longer length is not a multiple of the shorter one: `1:3 + 1:2` gives `2 4 4` with *longer object length is not a multiple of shorter object length*, while `1:4 + 1:2` gives `2 4 4 6` and says nothing. So the warning is not “you recycled”. It is “your recycling did not come out even”. The silent case is the one to watch.

1.6 Reading in data from external files

- Download `Cars.dat` from <https://raw.githubusercontent.com/boxiang-wang/stat5400/main/data/Cars.dat>.
- Use a text editor to look at this file. Note that the separators between columns are tabs (You can tell because the cursor jumps) and that the decimal point in numbers is indicated by periods.

- We need to read this file into an **object** in R to analyze it. R has several functions that read in data files in different formats.
- We will use R's built-in help facility to figure out which one to use.

On this page the file is already next to the notes, as `data/Cars.dat`. **Run this block first** — everything after it uses `Cars`.

```
Cars <- read.delim("data/Cars.dat")  
# <- is assignment operator  
str(Cars)
```

```
# show the first six rows  
head(Cars)
```

```
# show the last six rows  
tail(Cars)
```

```
summary(Cars)
```

```
plot(Cars)
```

1.7 Data frames

The object `Cars` is a data frame.

```
class(Cars)
```

Data frame:

- Special kind of object.
- Like a table of data.
 - Row for each observation.
 - Column for each variable.
- Variables (columns) can be of different data types:
 - Numeric, character, factors.

2 Data Structures

2.1 A question regarding proficiency test

Incoming graduate students used to take a **programming proficiency test** before the semester started. We no longer give it, but the two questions are still a good warm-up. The email below is about question 2.

Dear Dr. Boxiang,

For the second question in the proficiency test, we're asked to create an array containing 10 rows and 6 columns.

Is this equivalent to creating a matrix or a data.frame in r?

Dr. Boxiang's answer: not really.

See the table from Wickham's book:

	Homogeneous	Heterogeneous
1-dim	Atomic vector	List
2-dim	Matrix	Data frame
<i>n</i> -dim	Array	

2.2 Vectors

Atomic vector: same type — **integer**, **double**, **logical**, **character**.

```
a1 <- c(1, 2, 3)
class(a1)
typeof(a1)
is.integer(a1)
is.atomic(a1)
```

```
a2 <- c(1L, 2L, 3L)
# same to a2 <- seq(3) or seq.int(3)
class(a2)
typeof(a2)
```

```
a3 <- c(T, F, TRUE, FALSE) # ALWAYS spell T and F.
class(a3); typeof(a3) # semicolons not recommended.
as.numeric(a3)
```

💡 Predict — what does the second line give? This is why the comment above says to spell T and F out.

```
T <- FALSE
c(T, TRUE)
rm(T)      # put things back
T
```

i What just happened?

T is an ordinary variable; TRUE is not. T and F start out as shortcuts for TRUE and FALSE, but nothing stops you, a colleague, or a package from assigning to them. TRUE and FALSE are reserved words and cannot be reassigned.

So if `(x == T)` can silently mean the opposite of what you wrote. `rm(T)` deletes your version and uncovers the original.

```
a4 <- c("T", "F")
class(a4); typeof(a4)
as.numeric(a4)
```

```
a5 <- c(T, F, "T", "F")
typeof(a5)
```

i Going further: class, typeof, and mode

👤 **Three questions that sound the same and are not.** `typeof()` gives how the value is stored, `class()` gives what R dispatches methods on, and `mode()` is the older S-era grouping.

1	class numeric	typeof double	mode numeric
1L	class integer	typeof integer	mode numeric
TRUE	class logical	typeof logical	mode logical
"a"	class character	typeof character	mode character

The line that catches people is the first: a plain 1 is stored as a double, but its class is `numeric`. That is why `is.integer(1)` is FALSE while `is.integer(1L)` is TRUE, and why `1:3` (integer) and `c(1, 2, 3)` (double) are not the same vector.

2.3 Lists

The element of a list can be of different types.

```
l1 <- list(c(1, 2), c(T, F), c("T", "F"), 1 + 2)
str(l1)
unlist(l1)
```

```
l2 <- list(list(), list(list()))
str(l2)
```

This is a Matryoshka doll!

2.4 Three properties of vectors and lists

- **Name:**

```
a1 <- c(T, T, F)
l1 <- list(T, T, 2)
names(a1) <- names(l1) <- paste0("name", 1:3)
```

```
a1
```

```
l1$name2
l1[2]
```

- **Length:**

```
length(a1); length(l1)
```

- **Attributes:**

```
attr(a1, "my_attr") <- "This is my vector."
attributes(a1)
a1
```

2.5 Factors

Important use of attributes — **factors**.

```
x <- factor(c("S", "T", "A", "T"))
x
```

```
is.atomic(x)
is.factor(x)
```

```
attributes(x)
levels(x)
```

In the `Cars` data, `Country` is the natural factor. Since R 4.0, `read.delim` keeps text as `character`, so make the factor yourself.

```
is.factor(Cars$Country)
Cars$Country <- factor(Cars$Country)
levels(Cars$Country)
```

```
boxplot(Cars$MPG ~ Cars$Country)
```

💡 Predict — a factor that looks like numbers. What does the second line give?

```
f <- factor(c("10", "9", "8"))
as.numeric(f)
```

i Going further: never call `as.numeric` on a factor

👤 **You get 1 3 2, not 10 9 8.** A factor stores small integer codes plus a table of levels. `as.numeric()` hands back the codes, not the labels.

The levels are sorted as text, so they come out "10" "8" "9" — "10" sorts before "8" because "1" comes before "8". The codes then follow that order.

The safe idiom goes through the labels:

```
as.numeric(as.character(f)) # 10 9 8
```

Two more worth knowing. `factor(x, levels = ...)` sets the order yourself, which is how you control the order of boxes in the plot above. And after subsetting, unused levels stay behind: `droplevels()` removes them.

2.6 Matrices and arrays

- Vector + attribute(`dim`) = array.
- Two-dimensional array = matrix.

```
a <- matrix(1:6, ncol = 3, nrow = 2)
b <- array(1:12, c(2, 3, 2))
```

```
b <- 1:12
dim(b) <- c(2, 3, 2)
```

```
rownames(a) <- c("row1", "row2")
colnames(a) <- paste0("col", 1:3)
dimnames(b) <- list(letters[1:2], LETTERS[1:3],
  c("one", "two"))
```

```
# a
# b
```

2.7 Data frames

- Most common way to store data in R.
- *Behave* like a two-dimensional list.

```
dat <- data.frame(x1 = 2:4, x2 = letters[1:3])
str(dat)
```

Old R (before 4.0) converted strings to factors when creating data frames; you had to write `stringsAsFactors = FALSE` to keep them as text. Since R 4.0 that is the default, so the two blocks now agree.

```
dat2 <- data.frame(x1 = 2:4, x2 = letters[1:3],
  stringsAsFactors = FALSE)
str(dat2)
```

```
is.list(Cars)
dim(Cars)
```

```
is.data.frame(Cars)
str(Cars)
```

 Going further: a data frame is a list

 A data frame is a list of equal-length columns, wearing a two-dimensional coat. That is why `is.list(Cars)` above is `TRUE`.

It explains several things at once. `length(Cars)` is 8, the number of columns, not the number of rows. Anything that works on a list works here, so `sapply(Cars, class)` gives the type of every column in one line. And `Cars[1]` returns a one-column data frame while `Cars[[1]]` returns the column itself — the same `[` versus `[[` rule as any other list. The only thing the coat adds is the requirement that every column has the same length.

2.8 Summary of data structure

- Three properties of a vector or a list are **type**, **length**, and **attributes**.
- The four common types of atomic vectors are **integer**, **double**, **logical**, and **character** (strings).
- The elements of a **list** and a **data frame** can be of any type. The elements of an **atomic vector** and a **matrix** must be of the same type.

3 Subsetting

3.1 Subsetting a vector

- Positive integers:

```
(x <- 1:4 * 10)
x[c(1, 3)] # assign values: x[c(1, 3)] <- c(1, 2)
```

```
x[order(x)]
x[c(1, 1)]
x[c(1.2, 2.9)]
```

- Negative integers:

```
x[-c(3, 1)]
x[c(-3, 1)]
```

- Logical vectors:

```
x[x < 25]
which(x < 25)
```

```
x[c(TRUE, FALSE, TRUE, TRUE)]
x[c(TRUE, FALSE)]
```

```
x[c(TRUE, FALSE, TRUE, TRUE, TRUE)]
x[c(TRUE, FALSE, NA, TRUE)]
```

- **Characters:**

```
names(x) <- letters[1:4]
x[c("a", "c")]
```

👉 By hand — a chatbot was asked what `v[c(TRUE, FALSE)]` returns. Its answer is in the comments, and one claim in it is wrong. Run the block and check it against R. Done when you can say how many elements come back, and why. No AI for this block.

```
# Chatbot: "Logical subsetting keeps the elements marked TRUE. The index
# here has only two values, so R uses them for the first two elements and
# ignores the rest. v[c(TRUE, FALSE)] therefore returns the first element,
# a vector of length 1."
v <- 1:6 * 5
v[c(TRUE, FALSE)]
```

Solution

Three elements come back, not one: 5 15 25. The index is shorter than `v`, so R recycles it to `TRUE FALSE TRUE FALSE TRUE FALSE`. Nothing is “ignored”, which is the part the chatbot got wrong.

Recycling does not need the lengths to divide evenly. Change `v` to `1:7 * 5` and the same index returns elements 1, 3, 5, and 7, with no warning.

3.2 Subsetting a list

```
l1 <- list(a = 1, b = 2, c = 3)
```

```
l1$a
l1[[1]]
l1[1]
```

 Going further: two corners of [[

 [[with a vector digs down, it does not pick several. For a nested list, `l1[[c(2, 1)]]` means “element 2, then element 1 inside that” — one step per number. To pick several elements, use `[`.

You cannot mix positive and negative indices. `x[c(-1, 2)]` is an error, *only 0's may be mixed with negative subscripts*, because R cannot tell whether you meant to keep or to drop. Pick one direction.

If a list `x` is a train carrying objects, then `x[1]` is a train of car 1 and `x[[1]]` is the objects in car 1 — @RLangTip.

3.3 Words on the drop option

- Factor (drops excluded levels by `drop=TRUE`):

```
f1 <- factor(c("s", "t", "a", "t"))
f1[1:2]
f1[1:2, drop = TRUE]
```

- Matrix (drops dimension whose length is one):

```
a <- matrix(1:8, 2, 4)
colMeans(a[, 1:2])
a[, 1]
```

The next block is meant to fail: `a[, 1]` is no longer a matrix.

```
colMeans(a[, 1])
```

```
a[, 1, drop = FALSE]
colMeans(a[, 1, drop = FALSE])
```

4 A Simple Statistical Analysis

4.1 Subsetting rows and columns of a data frame

```
Cars[1:6, ]
```

```
head(Cars[, c("Country", "MPG", "Weight")])
```

```
head(Cars[Cars$Country == "U.S.", ])
```

```
head(Cars[Cars$Country == "Japan", ])
```

```
table(Cars$Country)
summary(Cars[Cars$Country == "U.S.", "MPG"])
summary(Cars[Cars$Country == "Japan", "MPG"])
```

```
hist(Cars[Cars$Country == "U.S.", "MPG"])
```

```
hist(Cars[Cars$Country == "Japan", "MPG"])
```

4.2 One-sample t-test

Try `help(t.test)` first.

```
t.test(Cars[Cars$Country == "U.S.", "MPG"])
```

```
UStout <- t.test(Cars[Cars$Country == "U.S.", "MPG"])
names(UStout)
```

```
UStout$conf.int
UStout$estimate
UStout$statistic
```

```
t.test(Cars[Cars$Country == "Japan", "MPG"])
```

4.3 Two-sample t-test

```
t.test(Cars[Cars$Country == "U.S.", "MPG"],  
       Cars[Cars$Country == "Japan", "MPG"])
```